

Chapter 23 — Ordered Set / Ordered Map

0. What You Will Learn in This Chapter

Ordered Set and Ordered Map are important C++ data structures used when you need both:

- sorted order
- dynamic updates

They are built on balanced binary search trees.

In C++, the main containers are:

set
map
multiset
multimap

These structures keep elements sorted automatically.

By the end of this chapter, you should understand:

- What ordered set and ordered map are
- How they differ from unordered set/map
- When to use ordered containers
- How lower_bound and upper_bound work
- How to handle duplicates with multiset
- How to maintain a dynamic sorted window
- How to solve calendar booking problems
- How to solve range overlap problems
- How to solve sliding window median
- How to use ordered maps for sweep line
- How to avoid iterator mistakes
- How ordered containers compare with heap and BST

Core problems covered in this chapter:

1. Contains Duplicate III
 2. My Calendar I
 3. My Calendar II
 4. My Calendar III
 5. Time Based Key-Value Store, ordered-map variation
 6. Sliding Window Median
 7. Data Stream as Disjoint Intervals
 8. Range Module
 9. Count of Smaller Numbers After Self, ordered concept
 10. Reverse Pairs, ordered concept
 11. Number of Flowers in Full Bloom
 12. Snapshot Array, map variation
 13. Exam Room
 14. Design a Food Rating System, set/map design
 15. Max Sum of Rectangle No Larger Than K
 16. Minimum Absolute Difference Queries, concept comparison
-

1. What Are Ordered Set and Ordered Map?

Ordered containers store elements in sorted order.

1.1 set

A set stores unique keys in sorted order.

```
set<int> st;  
  
st.insert(5);  
st.insert(1);  
st.insert(3);
```

The set stores:

1, 3, 5

1.2 map

A map stores key-value pairs sorted by key.

```
map<int, string> mp;  
  
mp[10] = "ten";  
mp[3] = "three";  
mp[7] = "seven";
```

Keys are stored in order:

3, 7, 10

1.3 multiset

A multiset stores sorted values and allows duplicates.

```
multiset<int> ms;  
  
ms.insert(5);  
ms.insert(5);  
ms.insert(2);
```

Stores:

2, 5, 5

2. Why Ordered Containers Exist

Hash maps and hash sets provide fast lookup, but no sorted order.

Example:

```
unordered_set<int> st;
```

Can answer:

Does x exist?

But cannot efficiently answer:

What is the smallest number $\geq x$?

What is the largest number $\leq x$?

What is the next interval after start?

What is the closest value to x?

Ordered containers answer these using:

lower_bound

upper_bound

in $O(\log n)$.

3. Ordered vs Unordered Containers

Internal structure	hash table	balanced BST
Order	no order	sorted order
Average lookup	$O(1)$	$O(\log n)$
Worst-case lookup	$O(n)$	$O(\log n)$
lower_bound	no	yes
range queries	no	yes
closest value	no	yes

Use unordered containers for fast exact lookup.

Use ordered containers for sorted dynamic operations.

4. Core Need / Why Use Ordered Containers?

Use ordered containers when you need:

17. dynamic insertion/deletion
18. sorted order
19. nearest value lookup
20. range overlap lookup
21. min/max access
22. duplicates in sorted order

- 23. sliding window median
 - 24. event sweep line with sorted coordinates
-

5. When to Use Ordered Set / Map

5.1 Need Closest Value

Examples:

- Contains Duplicate III
- closest value within window
- lower/upper neighbor

5.2 Need Dynamic Sorted Window

Examples:

- Sliding Window Median
- Contains Duplicate III

5.3 Need Intervals Sorted by Start

Examples:

- My Calendar I
- Range Module
- Data Stream as Disjoint Intervals

5.4 Need Sweep Line Events Sorted by Coordinate

Examples:

- My Calendar II
- My Calendar III
- Meeting Rooms
- Number of Flowers in Full Bloom

5.5 Need Duplicates Sorted

Use multiset.

Examples:

- Sliding Window Median
 - dynamic min/max with duplicates
-

6. When NOT to Use Ordered Containers

6.1 Only Need Exact Lookup

Use `unordered_set` or `unordered_map`.

6.2 Need Top K

Use heap or quickselect.

6.3 Need Prefix Sum Range Queries

Use prefix sum, Fenwick tree, or segment tree.

6.4 Need Order Statistics by Rank

C++ standard set cannot directly get kth element.

Use:

- Fenwick tree with coordinate compression

- segment tree
- policy-based data structure, non-standard
- sorted vector if static

6.5 Need Queue Behavior

Use queue or deque.

7. Mental Models

7.1 Dynamic Sorted Array

Think of set as a sorted array where insert/delete/search are $O(\log n)$.

7.2 Sorted Dictionary

Think of map as dictionary where keys are always sorted.

7.3 Neighbors

For a value x:

```
auto it = st.lower_bound(x);
```

gives first element $\geq x$.

Previous iterator gives largest element $< x$.

7.4 Timeline Events

map<int, int> can store sweep events:

```
time -> delta
```

Scanning map in key order gives active count.

8. Pattern Recognition Checklist

Ask:

25. Do I need sorted order dynamically?
26. Do I need nearest greater or nearest smaller?
27. Do I need lower_bound or upper_bound?
28. Do I need insert and delete while maintaining order?
29. Do duplicates matter?
30. Do I need sorted intervals?
31. Do I need overlap detection?
32. Do I need sweep events in sorted time order?
33. Is hash map insufficient because order matters?
34. Is heap insufficient because arbitrary deletion is needed?

If yes, ordered containers may apply.

9. C++ Tools Needed

9.1 set

```
set<int> st;
```

```
st.insert(x);
```

```
st.erase(x);
st.count(x);
st.find(x);
st.lower_bound(x);
st.upper_bound(x);
```

9.2 map

```
map<int, int> mp;

mp[key] = value;
mp.lower_bound(key);
mp.upper_bound(key);
mp.erase(key);
```

9.3 multiset

```
multiset<int> ms;

ms.insert(x);
ms.erase(ms.find(x)); // erases one copy
```

Important:

```
ms.erase(x);
```

erases all copies of x.

To erase one occurrence:

```
auto it = ms.find(x);
if (it != ms.end()) {
    ms.erase(it);
}
```

9.4 First and Last Elements

```
*st.begin();    // smallest
*st.rbegin();    // largest
```

For map:

```
mp.begin()->first;
mp.rbegin()->first;
```

9.5 Iterator Previous

```
auto it = st.lower_bound(x);

if (it != st.begin()) {
    auto prevIt = prev(it);
}
```

Do not call `prev(st.begin())`.

10. Core Templates

Template 1 — Lower Bound in Set

```
auto it = st.lower_bound(x);

if (it != st.end()) {
    // *it is first value >= x
}
```

Template 2 — Upper Bound in Set

```
auto it = st.upper_bound(x);

if (it != st.end()) {
    // *it is first value > x
}
```

Template 3 — Check Previous Neighbor

```
auto it = st.lower_bound(x);

if (it != st.begin()) {
    auto prevIt = prev(it);
    // *prevIt is largest value < x
}
```

Template 4 — Dynamic Sliding Window Set

```
set<int> window;

for (int i = 0; i < nums.size(); i++) {
    // use window

    window.insert(nums[i]);

    if (i >= k) {
        window.erase(nums[i - k]);
    }
}
```

For duplicates, use `multiset`.

Template 5 — Interval Overlap With Map

Intervals are [start, end).

```
map<int, int> intervals; // start -> end

auto next = intervals.lower_bound(start);

if (next != intervals.end() && next->first < end) {
    return false;
}

if (next != intervals.begin()) {
    auto prevIt = prev(next);

    if (prevIt->second > start) {
        return false;
    }
}

intervals[start] = end;
return true;
```

Template 6 — Sweep Line With Ordered Map

```
map<int, int> events;

events[start]++;
events[end]--;

int active = 0;

for (auto [time, delta] : events) {
    active += delta;
}
```

Template 7 — Multiset Median Window

```
multiset<int> low;
multiset<int> high;
```

Maintain:

- low contains smaller half
- high contains larger half
- low.size() >= high.size()

- size difference at most 1

11. Major Variations

Closest value	set lower_bound	Contains Duplicate III
Dynamic median	multiset/two multisets	Sliding Window Median
Calendar booking	map intervals	My Calendar I
Sweep line	map events	My Calendar II/III
Range intervals	map start -> end	Range Module
Stream intervals	map intervals	Summary Ranges
Snapshot values	map index -> value changes	Snapshot Array
Dynamic ranking	Fenwick/segment tree preferred	Count Smaller
Rectangle no larger than K	set prefix sums	Max Sum Rectangle

12. Full Problem Walkthrough 1 — LeetCode 220: Contains Duplicate III

12.1 Problem Summary

Given array nums, integers indexDiff and valueDiff.

Return true if there exist indices i and j such that:

```
abs(i - j) <= indexDiff
abs(nums[i] - nums[j]) <= valueDiff
```

12.2 How to Identify Ordered Set

Need maintain values within last indexDiff indices.

Need find if any value is close to current value.

Hash set cannot answer closest value.

Use ordered set.

12.3 Key Idea

For current value x , need previous value y such that:

$$x - \text{valueDiff} \leq y \leq x + \text{valueDiff}$$

Use:

`lower_bound(x - valueDiff)`

If found value is $\leq x + \text{valueDiff}$, answer true.

12.4 C++ Code

```
class Solution {
public:
    bool containsNearbyAlmostDuplicate(vector<int>& nums,
                                      int indexDiff,
                                      int valueDiff) {
        set<long long> window;

        for (int i = 0; i < nums.size(); i++) {
            long long x = nums[i];

            auto it = window.lower_bound(x - valueDiff);

            if (it != window.end() && *it <= x + valueDiff) {
                return true;
            }

            window.insert(x);

            if (i >= indexDiff) {
                window.erase(nums[i - indexDiff]);
            }
        }

        return false;
    }
};
```

12.5 Why Use long long?

Values can be near INT_MIN or INT_MAX.

Expression:

`x - valueDiff`

can overflow int.

Use long long.

12.6 Dry Run

```
nums = [1,5,9,1]
indexDiff = 2
valueDiff = 3
```

Window size keeps at most previous 2 values.
No pair satisfies both constraints.
Return false.

12.7 Complexity

Each insert/delete/search:

```
O(log k)
```

Total:

```
O(n log k)
```

Space:

```
O(k)
```

12.8 Common Mistakes

- Using unordered set
 - Not removing old index
 - Overflow with int
 - Removing before checking, causing missed valid pair
-

13. Full Problem Walkthrough 2 — LeetCode 729: My Calendar I

13.1 Problem Summary

Implement calendar booking.

A booking [start, end) is valid if it does not overlap existing bookings.

Return true if booked, false otherwise.

13.2 How to Identify Ordered Map

Need store intervals sorted by start.

For new interval, only possible overlaps are:

- next interval by start
- previous interval by start

Use map.

13.3 C++ Code

```
class MyCalendar {
private:
```

```

    map<int, int> intervals;

public:
    MyCalendar() {
    }

    bool book(int start, int end) {
        auto next = intervals.lower_bound(start);

        if (next != intervals.end() && next->first < end) {
            return false;
        }

        if (next != intervals.begin()) {
            auto prevIt = prev(next);

            if (prevIt->second > start) {
                return false;
            }
        }

        intervals[start] = end;

        return true;
    }
};

```

13.4 Overlap Rule

Intervals are half-open:

[start, end)

So:

[10,20) and [20,30)

do not overlap.

Overlap exists if:

previousEnd > start
nextStart < end

13.5 Complexity

Each booking:

$O(\log n)$

Space:

$O(n)$

14. Full Problem Walkthrough 3 — LeetCode 731: My Calendar II

14.1 Problem Summary

Implement calendar where double bookings are allowed, but triple bookings are not. Return false if booking causes triple overlap.

14.2 How to Identify Ordered Map

This is sweep line with dynamic events.

Use ordered map:

time -> delta

For booking:

events[start]++
events[end]--

Then scan active count.

If active reaches 3, undo and return false.

14.3 C++ Code

```
class MyCalendarTwo {
private:
    map<int, int> events;

public:
    MyCalendarTwo() {}

    bool book(int start, int end) {
        events[start]++;
        events[end]--;

        int active = 0;

        for (auto [time, delta] : events) {
            active += delta;

            if (active >= 3) {
                events[start]--;
            }
        }
    }
};
```

```

        events[end]++;

        if (events[start] == 0) {
            events.erase(start);
        }

        if (events[end] == 0) {
            events.erase(end);
        }

        return false;
    }
}

return true;
}
};

```

14.4 Complexity

Each booking scans events.

For n bookings:

$O(n)$

per booking after map updates.

Map insertion:

$O(\log n)$

Space:

$O(n)$

14.5 Common Mistakes

- Forgetting to undo failed booking
- Treating end as inclusive
- Not erasing zero-count events

15. Full Problem Walkthrough 4 — LeetCode 732: My Calendar III

15.1 Problem Summary

Each booking adds an event.

Return maximum number of overlapping events after each booking.

15.2 How to Identify Ordered Map

Same sweep-line events as My Calendar II.

But now triple booking is allowed.

Need maximum active count.

15.3 C++ Code

```
class MyCalendarThree {
private:
    map<int, int> events;

public:
    MyCalendarThree() {
    }

    int book(int start, int end) {
        events[start]++;
        events[end]--;

        int active = 0;
        int best = 0;

        for (auto [time, delta] : events) {
            active += delta;
            best = max(best, active);
        }

        return best;
    }
};
```

15.4 Complexity

Each booking:

$O(n)$

for scanning events.

Space:

$O(n)$

16. Full Problem Walkthrough 5 — LeetCode 981: Time Based Key-Value Store

16.1 Problem Summary

Design key-value store:

```
set(key, value, timestamp)
get(key, timestamp)
```

Return value with largest timestamp $\leq$ given timestamp.

16.2 How to Identify Ordered Map

Need floor lookup by timestamp.

For each key, store timestamps in sorted order.

Use:

```
unordered_map<string, map<int, string>>
```

16.3 C++ Code

```
class TimeMap {
private:
    unordered_map<string, map<int, string>> store;

public:
    TimeMap() {}

    void set(string key, string value, int timestamp) {
        store[key][timestamp] = value;
    }

    string get(string key, int timestamp) {
        if (!store.count(key)) {
            return "";
        }

        auto& timeline = store[key];

        auto it = timeline.upper_bound(timestamp);

        if (it == timeline.begin()) {
```



```
        return "";
    }

    --it;

    return it->second;
}
};
```

16.4 Complexity

Set:

$O(\log m)$

Get:

$O(\log m)$

where m is number of timestamps for key.

16.5 Note

Because LeetCode timestamps are strictly increasing per key, vector + binary search is often better.

But ordered map is a valid dynamic version.

17. Full Problem Walkthrough 6 — LeetCode 480: Sliding Window Median

17.1 Problem Summary

Given array and window size k , return median of each sliding window.

17.2 How to Identify Ordered Container

Need median dynamically as window slides.

Need insert and delete arbitrary outgoing value.

Heap has deletion issue.

Use two multisets.

17.3 Invariants

Use:

```
multiset<int> low; // smaller half
multiset<int> high; // larger half
```

Maintain:

```
low.size() == high.size()
or
low.size() == high.size() + 1
```

All values in low $\leq$ all values in high.

Median:

- odd k: largest of low
 - even k: average of largest low and smallest high
-

17.4 C++ Code

```
class Solution {
public:
    multiset<int> low;
    multiset<int> high;

    void rebalance() {
        while (low.size() > high.size() + 1) {
            auto it = prev(low.end());
            high.insert(*it);
            low.erase(it);
        }

        while (low.size() < high.size()) {
            auto it = high.begin();
            low.insert(*it);
            high.erase(it);
        }
    }

    void add(int x) {
        if (low.empty() || x <= *prev(low.end())) {
            low.insert(x);
        } else {
            high.insert(x);
        }

        rebalance();
    }

    void remove(int x) {
        auto it = low.find(x);
```

```

        if (it != low.end()) {
            low.erase(it);
        } else {
            it = high.find(x);
            high.erase(it);
        }

        rebalance();
    }

    double getMedian(int k) {
        if (k % 2 == 1) {
            return *prev(low.end());
        }

        return ((double)*prev(low.end()) + *high.begin()) / 2.0;
    }

    vector<double> medianSlidingWindow(vector<int>& nums, int k) {
        vector<double> result;

        for (int i = 0; i < nums.size(); i++) {
            add(nums[i]);

            if (i >= k) {
                remove(nums[i - k]);
            }

            if (i >= k - 1) {
                result.push_back(getMedian(k));
            }
        }

        return result;
    }
};

```

17.5 Complexity

Each add/remove:

$O(\log k)$

Total:

$O(n \log k)$

Space:

$O(k)$

17.6 Common Mistakes

- Erasing all duplicates with `erase(x)`
- Not rebalancing after deletion
- Integer overflow in median average
- Assuming outgoing value is always in one set

18. Full Problem Walkthrough 7 — LeetCode 352: Data Stream as Disjoint Intervals

18.1 Problem Summary

Given stream of integers, maintain disjoint intervals.

Implement:

```
addNum(value)
getIntervals()
```

18.2 How to Identify Ordered Map

Need intervals sorted by start.

When adding value, find neighboring intervals to merge.

Use map:

```
start -> end
```

18.3 C++ Code

```
class SummaryRanges {
private:
    map<int, int> intervals;

public:
    SummaryRanges() {
    }

    void addNum(int value) {
        auto next = intervals.upper_bound(value);

        int start = value;
```

```

int end = value;

if (next != intervals.begin()) {
    auto prevIt = prev(next);

    if (prevIt->second >= value) {
        return;
    }

    if (prevIt->second + 1 == value) {
        start = prevIt->first;
        intervals.erase(prevIt);
    }
}

if (next != intervals.end() && next->first == value + 1) {
    end = next->second;
    intervals.erase(next);
}

intervals[start] = end;
}

vector<vector<int>> getIntervals() {
    vector<vector<int>> result;

    for (auto [start, end] : intervals) {
        result.push_back({start, end});
    }

    return result;
}
};

```

18.4 Complexity

addNum:

$O(\log n)$

getIntervals:

$O(\text{number of intervals})$

19. Full Problem Walkthrough 8 — LeetCode 715: Range Module

19.1 Problem Summary

Design module tracking ranges.

Operations:

```
addRange(left, right)
queryRange(left, right)
removeRange(left, right)
```

Ranges are half-open:

```
[left, right)
```

19.2 How to Identify Ordered Map

Need maintain disjoint intervals sorted by start.

Use map:

```
start -> end
```

19.3 C++ Code

```
class RangeModule {
private:
    map<int, int> intervals;

public:
    RangeModule() {}

    void addRange(int left, int right) {
        auto it = intervals.lower_bound(left);

        if (it != intervals.begin()) {
            auto prevIt = prev(it);

            if (prevIt->second >= left) {
                it = prevIt;
            }
        }

        while (it != intervals.end() && it->first <= right) {
            left = min(left, it->first);
        }
    }
};
```

```

        right = max(right, it->second);

        auto eraselt = it;
        it++;
        intervals.erase(eraselt);
    }

    intervals[left] = right;
}

bool queryRange(int left, int right) {
    auto it = intervals.upper_bound(left);

    if (it == intervals.begin()) {
        return false;
    }

    --it;

    return it->second >= right;
}

void removeRange(int left, int right) {
    auto it = intervals.lower_bound(left);

    if (it != intervals.begin()) {
        auto prevIt = prev(it);

        if (prevIt->second > left) {
            it = prevIt;
        }
    }
}

vector<pair<int, int>> toAdd;

while (it != intervals.end() && it->first < right) {
    int start = it->first;
    int end = it->second;

    auto eraselt = it;
    it++;
    intervals.erase(eraselt);
}

```

```

        if (start < left) {
            toAdd.push_back({start, left});
        }

        if (end > right) {
            toAdd.push_back({right, end});
        }
    }

    for (auto [start, end] : toAdd) {
        intervals[start] = end;
    }
}
};

```

19.4 Complexity

Each operation:

$O(\log n + m)$

where m is number of overlapping intervals modified.

20. Full Problem Walkthrough 9 — Count of Smaller Numbers After Self, Ordered Concept

20.1 Problem Summary

For each element, count how many smaller elements appear after it.

Example:

nums = [5,2,6,1]

Output:

[2,1,1,0]

20.2 Why Standard Set Is Not Enough

Need count of values smaller than current.

C++ set does not support order statistics like:

number of elements $< x$

efficiently.

Need:

- Fenwick tree + coordinate compression
 - segment tree
 - policy-based data structure
 - merge sort counting
-

20.3 Lesson

Ordered set gives nearest values, but not rank counts in standard C++.
This belongs later to Fenwick / Segment Tree chapter.

21. Full Problem Walkthrough 10 — Number of Flowers in Full Bloom

21.1 Problem Summary

Given flowers with start and end bloom times, and people arrival times.
For each person, return number of flowers blooming at that time.
A flower blooms if:

$\text{start} \leq \text{time} \leq \text{end}$

21.2 Ordered Concept

Use sorted starts and sorted ends.
For time t :

$\text{started} = \text{count of start} \leq t$
 $\text{ended} = \text{count of end} < t$
 $\text{answer} = \text{started} - \text{ended}$

21.3 C++ Code

```
class Solution {
public:
    vector<int> fullBloomFlowers(vector<vector<int>>& flowers,
                                vector<int>& people) {
        vector<int> starts;
        vector<int> ends;

        for (auto& flower : flowers) {
            starts.push_back(flower[0]);
            ends.push_back(flower[1]);
        }

        sort(starts.begin(), starts.end());
        sort(ends.begin(), ends.end());
    }
};
```

```

vector<int> answer;

for (int time : people) {
    int started = upper_bound(starts.begin(), starts.end(), time) - starts.begin();
    int ended = lower_bound(ends.begin(), ends.end(), time) - ends.begin();

    answer.push_back(started - ended);
}

return answer;
};

```

21.4 Complexity

Sorting:

$O(n \log n)$

Each query:

$O(\log n)$

Total:

$O((n + q) \log n)$

22. Full Problem Walkthrough 11 — Snapshot Array, Map Variation

22.1 Problem Summary

Implement array with snapshots.

Operations:

```

set(index, val)
snap()
get(index, snap_id)

```

22.2 Ordered Map Idea

For each index, store changes by snapshot id.

Need get value at largest snap id $\leq$ requested.

Use map and upper_bound.

22.3 C++ Code

```

class SnapshotArray {
private:

```

```

int snapId;
vector<map<int, int>> history;

public:
    SnapshotArray(int length) {
        snapId = 0;
        history.resize(length);

        for (int i = 0; i < length; i++) {
            history[i][0] = 0;
        }
    }

    void set(int index, int val) {
        history[index][snapId] = val;
    }

    int snap() {
        snapId++;
        return snapId - 1;
    }

    int get(int index, int snap_id) {
        auto it = history[index].upper_bound(snap_id);

        --it;

        return it->second;
    }
};

```

22.4 Complexity

Set:

$O(\log m)$

Get:

$O(\log m)$

Snap:

$O(1)$

23. Full Problem Walkthrough 12 — LeetCode 855: Exam Room

23.1 Problem Summary

Students enter exam room with seats $0..n-1$.

Each student chooses seat maximizing distance to closest occupied seat.

If tie, choose smallest index.

Support:

```
seat()
leave(p)
```

23.2 Ordered Set Idea

Maintain occupied seats in sorted order.

For `seat()`, inspect gaps between adjacent occupied seats.

This is not the most optimized implementation, but it is clear.

23.3 C++ Code

```
class ExamRoom {
private:
    int n;
    set<int> seats;

public:
    ExamRoom(int n) {
        this->n = n;
    }

    int seat() {
        if (seats.empty()) {
            seats.insert(0);
            return 0;
        }

        int bestSeat = 0;
        int bestDist = *seats.begin();

        int prevSeat = *seats.begin();

        for (auto it = next(seats.begin()); it != seats.end(); ++it) {
            int currSeat = *it;
            int dist = (currSeat - prevSeat) / 2;
```

```

        if (dist > bestDist) {
            bestDist = dist;
            bestSeat = prevSeat + dist;
        }

        prevSeat = currSeat;
    }

    int lastDist = (n - 1) - *seats.rbegin();

    if (lastDist > bestDist) {
        bestSeat = n - 1;
    }

    seats.insert(bestSeat);

    return bestSeat;
}

void leave(int p) {
    seats.erase(p);
}
};

```

23.4 Complexity

Seat:

$O(\text{number of occupied seats})$

Leave:

$O(\log n)$

More optimized versions use interval priority structures.

24. Full Problem Walkthrough 13 — Design a Food Rating System

24.1 Problem Summary

Design food rating system supporting:

- change rating of food
- get highest-rated food for cuisine
- tie by lexicographically smallest food name

24.2 Ordered Set / Map Idea

For each cuisine, maintain ordered set of:

(-rating, foodName)

Why negative rating?

C++ set sorts ascending.

So highest rating becomes smallest negative number.

24.3 C++ Code

```
class FoodRatings {
private:
    unordered_map<string, string> foodCuisine;
    unordered_map<string, int> foodRating;
    unordered_map<string, set<pair<int, string>>> cuisineFoods;

public:
    FoodRatings(vector<string>& foods,
                vector<string>& cuisines,
                vector<int>& ratings) {
        for (int i = 0; i < foods.size(); i++) {
            string food = foods[i];
            string cuisine = cuisines[i];
            int rating = ratings[i];

            foodCuisine[food] = cuisine;
            foodRating[food] = rating;

            cuisineFoods[cuisine].insert({-rating, food});
        }
    }

    void changeRating(string food, int newRating) {
        string cuisine = foodCuisine[food];
        int oldRating = foodRating[food];

        cuisineFoods[cuisine].erase({-oldRating, food});

        foodRating[food] = newRating;

        cuisineFoods[cuisine].insert({-newRating, food});
    }

    string highestRated(string cuisine) {
```

```
        return cuisineFoods[cuisine].begin()->second;
    }
};
```

24.4 Complexity

Change rating:

$O(\log n)$

Highest rated:

$O(1)$

after set lookup.

25. Full Problem Walkthrough 14 — Max Sum of Rectangle No Larger Than K

25.1 Problem Summary

Given matrix and integer k, find maximum rectangle sum no larger than k.

25.2 Ordered Set Idea

Reduce 2D to 1D.

For 1D max subarray sum no larger than k:

Need maximize:

$\text{prefix}[i] - \text{prefix}[j] \leq k$

So need:

$\text{prefix}[j] \geq \text{prefix}[i] - k$

Use ordered set of previous prefixes and lower_bound.

25.3 1D Helper

```
int maxSubArrayNoMoreThanK(vector<int>& nums, int k) {
    set<int> prefixes;
    prefixes.insert(0);

    int prefix = 0;
    int best = INT_MIN;

    for (int x : nums) {
        prefix += x;
```

```

    auto it = prefixes.lower_bound(prefix - k);

    if (it != prefixes.end()) {
        best = max(best, prefix - *it);
    }

    prefixes.insert(prefix);
}

return best;
}

```

25.4 Full C++ Code

```

class Solution {
public:
    int maxSubArrayNoMoreThanK(vector<int>& nums, int k) {
        set<int> prefixes;
        prefixes.insert(0);

        int prefix = 0;
        int best = INT_MIN;

        for (int x : nums) {
            prefix += x;

            auto it = prefixes.lower_bound(prefix - k);

            if (it != prefixes.end()) {
                best = max(best, prefix - *it);
            }

            prefixes.insert(prefix);
        }

        return best;
    }

    int maxSumSubmatrix(vector<vector<int>>& matrix, int k) {
        int rows = matrix.size();
        int cols = matrix[0].size();

        int answer = INT_MIN;
    }
}

```



```

for (int left = 0; left < cols; left++) {
    vector<int> rowSum(rows, 0);

    for (int right = left; right < cols; right++) {
        for (int r = 0; r < rows; r++) {
            rowSum[r] += matrix[r][right];
        }

        answer = max(answer, maxSubArrayNoMoreThanK(rowSum, k));

        if (answer == k) {
            return k;
        }
    }
}

return answer;
}
};

```

25.5 Complexity

Let rows = R, cols = C.

Time:

$O(C^2 * R \log R)$

Can choose smaller dimension for squared loop.

Space:

$O(R)$

26. Mini Problem 1 — Erase One Element From Multiset

```

multiset<int> ms;

auto it = ms.find(x);

if (it != ms.end()) {
    ms.erase(it);
}

```

Do not use:

```
ms.erase(x);
```

if only one copy should be removed.

27. Mini Problem 2 — Floor and Ceiling

```
set<int> st;  
  
auto ceillt = st.lower_bound(x);  
  
if (ceillt != st.end()) {  
    int ceiling = *ceillt;  
}  
  
if (ceillt != st.begin()) {  
    int floorValue = *prev(ceillt);  
}
```

28. Mini Problem 3 — First Interval Starting After X

```
map<int, int> intervals;  
  
auto it = intervals.upper_bound(x);
```

This gives first interval with start > x.

29. Mini Problem 4 — Interval Containing Point

```
auto it = intervals.upper_bound(x);  
  
if (it != intervals.begin()) {  
    --it;  
  
    if (it->first <= x && x < it->second) {  
        // x is inside [it->first, it->second)  
    }  
}
```

30. Common Edge Cases

30.1 Empty Set

Check before dereferencing:

```
if (!st.empty())
```

30.2 Iterator at Begin

Do not call `prev(begin())`.

30.3 Iterator at End

Do not dereference `end()`.

30.4 Duplicates

Use `multiset`.

30.5 Erasing While Iterating

Save iterator before erase or use returned next iterator pattern.

30.6 Half-Open Intervals

Calendar problems usually use:

```
[start, end)
```

So end touching start is allowed.

30.7 Overflow

Use `long long` for differences near `int` boundaries.

31. Common Mistakes and Wrong Code

Mistake 1 — Using `unordered_set` When Order Is Needed

No `lower_bound`.

Use `set`.

Mistake 2 — Erasing All Duplicates From `Multiset`

Wrong:

```
ms.erase(x);
```

Correct:

```
ms.erase(ms.find(x));
```

Mistake 3 — Dereferencing `end()`

Wrong:

```
auto it = st.lower_bound(x);  
int y = *it;
```

Need:

```
if (it != st.end())
```

Mistake 4 — Calling prev(begin())

Always check:

```
if (it != st.begin())
```

Mistake 5 — Wrong Overlap Rule

For [start, end):

Overlap if:

```
prevEnd > start  
nextStart < end
```

Mistake 6 — Forgetting to Undo Failed Calendar Booking

In My Calendar II, undo event changes if triple booking happens.

32. Interview Explanation Templates

32.1 Ordered Set for Closest Value

I use an ordered set because I need the nearest value greater than or equal to a bound. `lower_bound` gives that in $O(\log n)$, while a hash set cannot provide ordered lookup.

32.2 Calendar Booking

I store existing intervals sorted by start time. For a new interval, only the previous and next intervals can overlap. I check those using `lower_bound`.

32.3 Sweep Line Map

I store start events as +1 and end events as -1 in an ordered map. Scanning the map in sorted time order gives the active count at each point.

32.4 Sliding Window Median

I maintain two multisets representing the lower and upper halves of the current window. This allows insertion, deletion, and median retrieval efficiently.

32.5 Range Module

I store disjoint intervals in an ordered map. To add, remove, or query a range, I find overlapping intervals using `lower_bound` and merge or split as needed.

33. Complexity Patterns

Set insert/delete/search	$O(\log n)$	$O(n)$
Map insert/delete/search	$O(\log n)$	$O(n)$
Contains Duplicate III	$O(n \log k)$	$O(k)$
My Calendar I	$O(\log n)$	$O(n)$
My Calendar II	$O(n)$ per booking	$O(n)$
My Calendar III	$O(n)$ per booking	$O(n)$
TimeMap map version	$O(\log m)$ per op	$O(\text{total sets})$
Sliding Window Median	$O(n \log k)$	$O(k)$
Summary Ranges	$O(\log n)$ add	$O(n)$
Range Module	$O(\log n + m)$	$O(n)$
Snapshot Array	$O(\log m)$ get/set	$O(\text{total changes})$
Food Ratings	$O(\log n)$ update	$O(n)$
Max Sum Rectangle $\leq K$	$O(C^2 * R \log R)$	$O(R)$

34. Time-Boxed Practice Guidance

Intro Ordered Containers

Target:

15–25 minutes

Examples:

- Contains Duplicate III
- My Calendar I
- TimeMap map version

Medium Ordered Container Problems

Target:

30–50 minutes

Examples:

- Sliding Window Median
- Summary Ranges
- My Calendar II
- My Calendar III
- Snapshot Array

Hard Ordered Interval Problems

Target:

60–90 minutes

Examples:

- Range Module
 - Max Sum of Rectangle No Larger Than K
 - Exam Room
 - Food Rating System
-

35. Expanded Practice Set

35.1 Must-Do Medium

- 35. Contains Duplicate III
- 36. My Calendar I
- 37. My Calendar II
- 38. My Calendar III
- 39. Time Based Key-Value Store
- 40. Sliding Window Median
- 41. Data Stream as Disjoint Intervals
- 42. Snapshot Array

35.2 Strong Interview Practice

- 43. Range Module
- 44. Exam Room
- 45. Design a Food Rating System
- 46. Number of Flowers in Full Bloom
- 47. Max Sum of Rectangle No Larger Than K
- 48. Count of Smaller Numbers After Self, Fenwick preferred
- 49. Reverse Pairs, merge sort/Fenwick preferred

35.3 Hard / Stretch

- 50. Range Module
 - 51. Sliding Window Median
 - 52. Max Sum of Rectangle No Larger Than K
 - 53. Exam Room
 - 54. Count of Range Sum
 - 55. Count of Smaller Numbers After Self
 - 56. Falling Squares, coordinate compression/segment tree later
-

36. Suggested Study Order

Step 1 — Basic Ordered Lookup

Start with:

- 57. Contains Duplicate III
- 58. Floor/Ceiling practice
- 59. TimeMap map variation

Goal:

Learn `lower_bound`, `upper_bound`, and iterator safety.

Step 2 — Interval Booking

Solve:

- 60. My Calendar I
- 61. My Calendar II
- 62. My Calendar III

Goal:

Learn interval overlap and sweep line maps.

Step 3 — Dynamic Sorted Window

Solve:

- 63. Sliding Window Median

Goal:

Learn multiset and duplicate-safe deletion.

Step 4 — Dynamic Intervals

Solve:

- 64. Summary Ranges
- 65. Range Module

Goal:

Learn interval merge, split, and query.

Step 5 — Design Problems

Solve:

- 66. Snapshot Array
- 67. Food Rating System
- 68. Exam Room

Goal:

Use ordered containers in object-oriented designs.

37. Pattern Decision Table

nearest value $\geq x$	lower_bound	Contains Duplicate III
nearest value $> x$	upper_bound	TimeMap
dynamic sorted window	multiset	Sliding Window Median
duplicates sorted	multiset	Median window
interval overlap	map start -> end	My Calendar I
sweep active count	map events	My Calendar II/III
disjoint intervals	map intervals	Summary Ranges

Problem Clue	Ordered Container Technique	Example
range add/remove/query	map intervals	Range Module
historical value $\leq$ time	map timestamp $\rightarrow$ value	Snapshot Array
highest rating with tie	set of pairs	Food Ratings
max subarray $\leq k$	set prefix sums	Max Sum Rectangle

38. Final Cheat Sheet

Set Lower Bound

```
auto it = st.lower_bound(x);

if (it != st.end()) {
    // *it >= x
}
```

Previous Value

```
if (it != st.begin()) {
    auto prevIt = prev(it);
}
```

Multiset Erase One Copy

```
auto it = ms.find(x);

if (it != ms.end()) {
    ms.erase(it);
}
```

Calendar Overlap

```
auto next = intervals.lower_bound(start);

if (next != intervals.end() && next->first < end) {
    return false;
}

if (next != intervals.begin()) {
    auto prevIt = prev(next);

    if (prevIt->second > start) {
```



```
        return false;
    }
}
```

Sweep Events

```
events[start]++;
events[end]--;

int active = 0;

for (auto [time, delta] : events) {
    active += delta;
}
```

TimeMap Floor Lookup

```
auto it = mp.upper_bound(timestamp);

if (it == mp.begin()) {
    return "";
}

--it;

return it->second;
```

39. What to Master Before Chapter 24

Before moving to Graph Representation, make sure you can:

- Use set
- Use map
- Use multiset
- Explain ordered vs unordered containers
- Use lower_bound
- Use upper_bound
- Safely use prev(it)
- Avoid dereferencing end()
- Erase one copy from multiset
- Solve Contains Duplicate III
- Solve My Calendar I
- Understand sweep line map events
- Solve Sliding Window Median with multiset
- Maintain disjoint intervals with map

-
- Know when ordered containers are not enough for rank queries
-

40. Core Takeaway

Ordered Set and Ordered Map are for dynamic sorted data.

Use them when you need:

insert/delete/search + sorted order

The most important operations are:

lower_bound

upper_bound

prev

begin

rbegin

Ordered containers are slower than hash containers for exact lookup, but they provide sorted navigation.

The key question is:

Do I need the nearest value, next interval, previous interval, or sorted dynamic window?

If yes, ordered containers are probably useful.
